Chapter 2

Introduction to R

This chapter introduces R as a statistical computing environment and familiarizes you with RStudio as an integrated development interface for working efficiently with data. It covers the fundamentals of working with R as well as data management, including how to import, manipulate, and organize data sets for analysis.

For years, a textbook chapter like this pointed towards tutorial websites, R textbooks, and Stack Overflow as the best way to get help with R. That advice is now largely outdated since AI models such as Claude, ChatGPT, and Gemini can typically get you an answer much faster and more tailored to your exact problem. Previously, you had to search tutorial site or go through a decade-old forum thread written for different package versions. The number of questions asked on Stack Overflow has collapsed since the advent of AI (Figure 2.1). If you obtain an error while using R, do not know which function to use, or want a plain-English explanation of a block of code, asking an AI assistant is now usually the right first move.

Besides requiring knowledge about statistics and data analysis in general, using AI effectively for R still requires additional skills and care. For example, you have to give AI the context and not just a question. Paste the actual error message and the output from R rather than describing the problem from memory. Giving AI details about the context you are working in, produces a concrete and usable answer. Ask AI to explain its approach and output besides simply producing an answer. Requesting code without asking for an explanation teaches you very little. So follow up with "explain this line" or "why this function instead of that one" to make sure the exercises at the end of the chapters still builds your own understanding of R. Probably the most important skill of using AI is to verify everything. AI can confidently invent functions or arguments that do not exist, or produce code written for an older, now-deprecated version of a package. Always run the code and check that the output makes sense

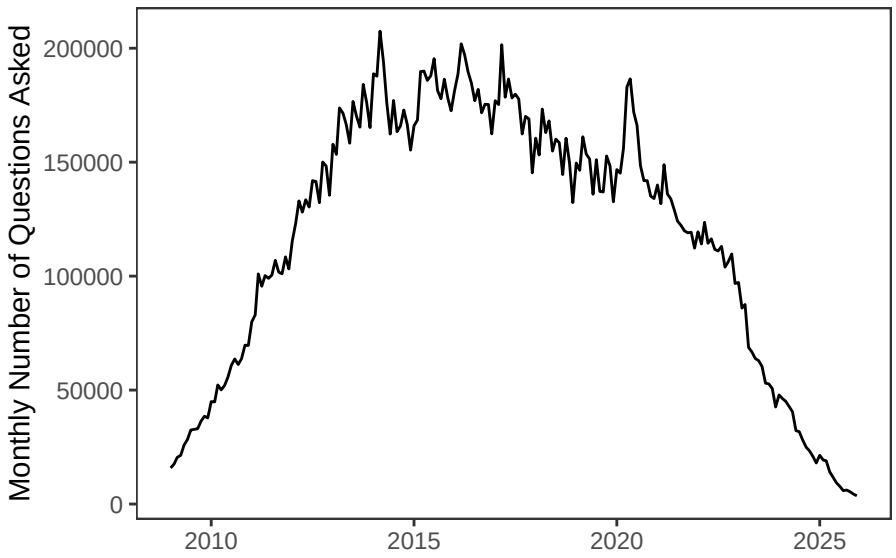

Figure 2.1: Monthly number of questions asked to Stack Overflow. Note the spike in 2020 with the beginning of the COVID-19 pandemic as well as the decline after the introduction of ChatGPT in November 2022.

before you trust it. This is true in the university setting and certainly in a professional environment where money is involved. If you are uncertain about the output from AI, you can use the official manual as a backup. That is, when you are unsure whether an argument the AI suggested is real, or need the precise definition of what a function returns, the R manual is the complete reference for every function in R and its packages. It includes worked examples at the bottom of most function pages (for instance, boxplot). And keep in mind that none of this replaces learning R and data analysis. An AI assistant should function like a good teaching assistant, i.e., it gets you unstuck quickly and helps you understand why, not do the thinking for you.

2.1 Opening RStudio

Work in RStudio is done in four windows:

1. Script Window
   - This is were you type your R Script (.R) and where you execute commands.
   - Comparable to do-file/editor in Stata.
   - This window needs to be opened by File ⇒ New File ⇒ R Script.
2. Console window
   - Use of R interactively. Should only be used for quick calculations and

not part of an analysis.

3. Environment
   - Lists all the variables, data frames, and user-created functions.
   - It is tempting to use the "Import Dataset" function . . . Don't.
4. Plots/Packages/Help

There is a base version of R that allows doing many calculations but the power of R comes through its packages, which are expanded and updated on a regular basis. To use functions associated with a particular package, you have to install the package first and then activate it. Click "Install" in the "Packages" window of RStudio and type in the name of the desired package. The installation has to be done only once. However, to use a package, you have to activate it every time you restart RStudio. Thus, the easiest way is to activate all the packages you use at the beginning of your script with the following command:

```
library(ggplot2)
```

Here, the package ggplot2 is used as an example. The package is used extensifly throughout this book for graphing.

The `#` allows you to include comments in your script file that are not executed by R. It is good practice to start any new script with clearing the memory using the command `rm(list=ls())`. Use the command `getwd()` to determine the current working directory or set the new working directory with the command `setwd()`, e.g., `setwd("E:/")`. For file paths, replace \ with /. Next, you want to load all libraries necessary for your entire script file with the command `library()` as mentioned before. It is also good practice to save your R-script on a regular basis. The frontmatter, i.e., the top of a R-script file, could look as follows

```
rm(list=ls())
load(url(paste("https://github.com/jrfdumortier/DataAnalysis",
               "/raw/main/DataAnalysisPAData.RData",sep="")))
library(openxlsx)
```

We break the URL into two separate string pieces purely for typesetting purposes, i.e., so the line of code does not run off the edge of the page. It is not required by R at all and would work exactly the same written as one long line. We split it this way because a single quoted string in R cannot be split across lines without the line break itself becoming part of the string. The extra newline (and any leading whitespace) would corrupt the URL and make `url()` fail. By writing it as two shorter, complete strings instead, the line break falls between them, outside any quotes, which is a safe place for R to continue reading the expression on the next line.

R lets you break a statement onto a new line anywhere the expression is still obviously incomplete, e.g. right after a comma, an open parenthesis, or an operator like a plus or a minus sign. Putting the break right after the comma in the above example separates the two string arguments here. The `paste()`

function then concatenates those two text pieces into a single character string. The `sep=""` argument tells it to join them with no character in between (rather than the default single space), so the result is the exact, unbroken URL you would get if you typed it all on one line.

2.2 Functions

At the core of R are functions that "do things" based on your input. In the previous section, we have already used a couple of functions like `library()`, `load()`, `url()`, or `paste()`. The basic structure is

```
object = functionname(argument1=value,argument2=value,...)
```

The structure has the following components

- `object`: Output of the function will be assigned to object.
- `functionname`: Name of the system function. You can also create and use your own functions. More about this later.
- `argument`: Arguments are function specific.
- `value`: The value you want a particular argument to take.

If a function is executed without an specific assignment, the output will be displayed in the console. Before using a function, read the documentation since you need to be aware of the default settings and values. In most cases, those defaults are set to values that satisfy most uses. For example, consider the help file for the function t.test: `t.test(x,y=NULL,...,mu=0,conf.level=0.95,...)`. For this function we have the following default values: `y=NULL`, `mu=0`, and `conf.level=0.95`.

2.3 Data in R

The main data types which can appear in the Environment window of R are vectors, matrices, data frames, and lists. Vectors are one-dimensional data structures that store elements of the same type (e.g., all numeric or all character) and form the basic building blocks for most objects in R.

```
preselection   = seq(1788,2016,4)
midterm        = seq(by=4,to=2018,from=1790)
```

Matrices are two-dimensional structures with rows and columns, where all elements must be of the same type, making them useful for mathematical and linear algebra operations. Note that only numerical values are allowed.

```
somematrix     = matrix(8,10,4)
```

Data frames are tabular data structures where each column can have a different data type, but all columns have the same length, making them ideal for representing data sets with multiple variables. Data frames are by far the most common

data type in R and are comparable to Excel spreadsheets. Lists are flexible containers that can hold elements of different types and lengths, including other lists, making them useful for organizing complex or hierarchical data.

```
myfirstlist = list(preselection,midterm,somematrix)
```

2.3.1 Using R as a Calculator

Entering heights of people and storing it in a vector named `height`:

```
height = c(71,77,70,73,66,69,73,73,75,76)
```

Calculating the sum, product, natural log, mean, and (element-wise) squaring is done with the following commands:

```
sum(height)
prod(height)
log(height)    # Default is the natural log
meanheight     = mean(height)
heightsq       = height^2
```

Removing (i.e., deleting) unused elements from the environment can be done as follows:

```
rm(myfirstlist)
```

Data frames are the most commonly used tables in R/RStudio. They are similar to an Excel sheet. Column names represent the variables and rows represent observations. Note that column names must be unique and without spaces.

```
studentid      = 1:10
name           = c("Andrew","Linda","William","Daniel","Gina",
                   "Mick","Sonny","Wilbur","Elisabeth","James")
students       = data.frame(studentid,name,height)
rm(studentid,height,name)
```

Indexing refers to identifying elements in your data. For most objects, we have the following

```
dataframe[row number,coloumn number]
```

For example, consider the following indexing exercises with the data frame `students`:

```
students[3,2]
students[c("name")]           # For selecting a certain column
students[c("name","height")]  # For selecting multiple columns
students[3,]                  # All information in the third row
```

Referring to a particular column in a data frame is done through the dollar

symbol. For example, you can use the following to return all the students' heights:

```
students$english
```

You will use this functionality very often.

To extract variables or observations based on certain criteria, the command `subset()` must be used. Consider the data `vehicles`. Extracting vehicle information only for the year 2015 is done as follows.

```
cars2015 = subset(vehicles,year==2015)
```

Note that the double equal sign conducts a logical test. Using a single equal sign does not extract any data and simply returns the original data without (!) an error message. To list all the distinct values in a column, the command `unique()` can be used. This command only makes sense in the case of categorical data in a particular column. For example, listing all EPA vehicle size classes ($VClass$) can be accomplished as follows.

```
unique(cars2015$vclass)
```

Suppose you are only interested in the variables *co2tailpipegpm* and *vclass* for the model year 2015.

```
cars2015 = subset(vehicles,year==2015,
                  select=c("co2tailpipegpm","vclass"))
```

Suppose you are only interested in "Compact Cars" and "Large Cars" in the column $VClass$ for the year 2015. There the notation is a bit odd (note that the many line breaks are not necessary to include in R):

```
cars2015 = subset(vehicles,
                  year==2015 & vclass %in% c("Compact Cars",
                                             "Large Cars"),
                  select=c("make","co2tailpipegpm","vclass"))
```

To aggregate data based on a function, e.g., sum or mean:

```
cars2015 = aggregate(co2tailpipegpm~make+vclass,
                     FUN=mean,data=cars2015)
```

2.3.2 Importing Data into R

In almost all cases, the data is imported into R from an external data set. The data has to be "machine-readable" which means that the first row must contain the variable names and the actual data starts in the second row. Machine-readable data can be imported as follows:

- `read.csv("filename.csv")`: If you have a comma separated value (.csv) file then this is the easiest and preferred way to import data.

- `readWorkbook(file="filename.xlsx",sheet="sheet name")`: Requires the package openxlsx. Note that there are many packages reading Excel and this is one of the most reliable and user-friendly.
- Importing data from other software packages (e.g., SAS, Stata, Minitab, SPSS) or .dbf (database) files can be achieved using the package foreign. The package works also for Stata data up to version 12. To important data from Stata version 13 and above, the package readstata13.

2.3.3 Exporting a Data Frame to .csv-File

To write data as a comma separated value (csv) file into the current working directory, you can use the following function:

```
write.csv(cars2015,"cars2015.csv",row.names=FALSE)
```

Using the option `row.names=FALSE` avoids an index column in the output file.

2.3.4 Reshaping Data from Long to Wide and Viceversa

R has the ability to reshape data from long to wide format and back. For this demonstration, we use the data in `compactcars` and the command `reshape()` from the package reshape2:

```
cars = melt(compactcars,
            id=c("year","make","model","displ","drive"))
```

Reshaping the data is generally very useful but also tricky. For detailed information see the section How can I reshape my data in R.

2.3.5 Extending the Basic `table()` Function

The required package to extend the basic `table()` function is called gmodels. Compare the outputs of the functions `table()` and `CrossTable()`. The commands below do the following:

- Standard `table()` function.
- The function `CrossTable()` includes the proportions along the two dimensions of gun ownership and gender. Note that a description of the cell content is at the top of the results page.

```
df        = subset(gss,year==2022,
                   select=c("owngun","sex"))
table(df$owngun,df$sex)
```

```
##
##       1   2
##   1 424 331
##   2 623 879
##   3  27  17
```

```
CrossTable(df$owngun,df$sex,prop.chisq=FALSE)
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |           N / Row Total |
## |           N / Col Total |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  2301
##
##
##                | df$sex
##    df$owngun |         1 |         2 | Row Total |
## -------------|-----------|-----------|-----------|
##            1 |       424 |       331 |       755 |
##              |     0.562 |     0.438 |     0.328 |
##              |     0.395 |     0.270 |           |
##              |     0.184 |     0.144 |           |
## -------------|-----------|-----------|-----------|
##            2 |       623 |       879 |      1502 |
##              |     0.415 |     0.585 |     0.653 |
##              |     0.580 |     0.716 |           |
##              |     0.271 |     0.382 |           |
## -------------|-----------|-----------|-----------|
##            3 |        27 |        17 |        44 |
##              |     0.614 |     0.386 |     0.019 |
##              |     0.025 |     0.014 |           |
##              |     0.012 |     0.007 |           |
## -------------|-----------|-----------|-----------|
## Column Total |      1074 |      1227 |      2301 |
##              |     0.467 |     0.533 |           |
## -------------|-----------|-----------|-----------|
##
##
```

Note that for almost any R command, you can store the output by assigning it to an object:

- `somename = CrossTable(gssgun$owngun,gssgun$sex,prop.chisq=FALSE)`

2.3.6 Merging Datasets

Consider two data sets from school districts in Ohio:

- `ohioscore` contains an identifier column IRN and a score that indicates the quality of the school.
- `ohioincome` contains the same identifier than the previous sheet in addition to median household income and enrollment.

To merge the two data frames based on the column IRN, the function `merge()` must be used:

```
ohioschool = merge(ohioscore,ohioincome,by=c("irn"))
```

2.4 Exercises

1. ***COVID-19 Airline Impact*** (***): Subset the `airlines` data for the years 2015–2023 and American Airlines, Delta, and United. Next, aggregate the number of arriving flights of those three airlines by year. Plot the total number by airline in one graph. Put the year on the horizontal axis and the number of flights on the vertical axis. The three airlines should be differentiated by different colors. What is the percentage decline in arriving flights from 2019 to 2020?

2. ***Motor Trend Cars*** (***): Use the `mtcars` data set which comes with R. Create a new variable in `mtcars` called `gpm` that expresses fuel consumption in gallons per mile. Next add a variable to the data frame called `efficiency` defined as `hp/gpm`. Report the top and bottom 5 cars based on this efficiency measure.

3. ***Vehicle Age and Income*** (***): Consider the data `nhtsveh` from the 2022 National Household Travel Survey (NHTS). There are two variables of interest to you for this question: `vehage` and `hhfaminc`. Check out the code book associated with the survey and the two variables in question. After eliminating all the data which would distort the results, calculate the average age of vehicles by income group. Are there any policy implications that you can think of that are related to the observed result?

4. ***EU Crime Decline*** (***): Consider the data in `eucrime`. Specifically, you should focus on intentional homicides per 100,000 inhabitants. From the five countries with the highest homicide rates in 2008, which had the largest decline over the time period 2008-2022.

5. ***Recidivism*** (***): Consider the data in `rossi`. We are only interest in three variables, i.e., `arrest`, `fin`, `married`. What is the percentage of those who were arrested that were married? You do need to differentiate your answer also based on `fin`. Explain why splitting by `fin` might matter here

6. ***Affordability across U.S. States*** (***): Using the data `affordability`, focus on 2025 and compute the ratio of *Average Monthly Rent* to *Average Weekly Earnings* (after converting weekly earnings to a monthly value) for each state. Identify the five states with the highest and lowest values of this ratio. Briefly interpret what a high or low ratio suggests about housing affordability in those states.

7. ***Cost Burden and Financial Stress*** (***): Affordability problems may be linked to financial stress. Using the data `affordability` for a given year, calculate the share of *Annual Childcare Cost* relative to annual earnings for each state. Create a scatter plot comparing this childcare cost share to one financial stress indicator of your choice (e.g., *Creditcard Delinquency Rate*, *Autoloan Delinquency Rate*, or *Student Loan Default Rate*). Describe the relationship you observe. Does higher cost burden appear to be associated with higher financial stress?

8. ***Changes in Affordability Over Time*** (***): Affordability is not static and can change over time. Choose one affordability-related item (e.g., Average Monthly Rent, Home Sale Prices, or Average Weekly Earnings) from the data `affordability`. For each state, compute the percentage change in this item between the first and last year available in the data. Plot these changes across states and identify which states experienced the largest increases or decreases. Briefly discuss what these changes imply about the evolution of affordability over time.

9. ***Long-Run Land-Use Change by Category in Brazil*** (***): Land use evolves in response to economic, environmental, and policy forces. For this question, use the data in `biomasbrazil`. First, aggregate the agricultural land use (i.e., *Soybean*, *Sugar Cane*, *Rice*, *Coffee*, *Citrus*, *Palm Oil*, *Cotton*, and *Other Agricultural Land*) into a category called *Cropland*. For each land-use category, calculate the total land area across all states in each year. Include *Cropland* as the category to summarize the crops. Plot the evolution of these totals over time for at least five selected land-use categories (including forest and one agricultural crop of your choice). Identify which land-use categories show the largest increases and decreases over the sample period, and briefly describe the main patterns you observe.

10. ***State-Level Transitions Over Time in Brazil*** (***): Changes in land use may differ substantially across states. Select one state and two land-use categories (for example, forest and pasture, or soybean and other agricultural land) in the data set `biomasbrazil`. Plot the land area in each category over time for that state. Describe whether the changes suggest stability, gradual transition, or rapid structural change, and discuss what this might imply about land-use dynamics in that state.

11. ***Seasonality in U.S. Power Generation*** (***): Electricity generation varies systematically over the year. For each power source (e.g., coal, nuclear, wind, solar) listed in `eianetgeneration`, compute the average

monthly generation across all years. Plot average generation by month for each source. Describe the seasonal patterns you observe and explain why some sources exhibit stronger seasonality than others.

12. ***U.S. Energy Transition*** (***): The U.S. electricity mix has changed substantially over time. For each year in the data set `eianetgeneration`, calculate the share of total electricity generation accounted for by each power source. Plot the evolution of these shares over time for at least four sources. Identify which sources are gaining or losing importance and briefly discuss what these trends suggest about the U.S. energy transition.